

vllm-xtu-moe

在廉价 CPU + GPU 异构设备上运行超大 MoE 模型

开源、可改造、可观测的混合推理实现

2×A100-40GB + 2×EPYC 9654 • C=4 聚合 100.16 tok/s • 同机专有引擎的 1.15×

28.13 ms

C=1 TPOT

59.89 t/s

C=2 聚合

100.16 t/s

C=4 聚合

刘锋 · 2026-09-15

中国开源软件协会

01

为什么做

算力成本与可研究性，决定了这条技术路线。

vllm-xtu-moe · 刘锋 · 2026-09-15

同一代模型，设备门槛相差一个数量级

目标是单人和小组“够用”，不是追求集群榜单。

300-1400万

H200/B300 生产级

30-50万

2x A100-40GB，本项目实测
RTX Pro 6000 性能最佳

≈1/10

显存规模与成本量级

可用标准

单流 20–40 tok/s
并发 60–150 tok/s

目标用户

科研、教学、小型实体应用
以及高信息安全要求的私有化部署

把专家权重放在 CPU / DRAM，价值不只是省显存

它让路由、激活和安全检查进入可观测的主机路径。

GPU-only / 典型 offload

CPU 主要承担显存扩展

权重按需搬到 GPU
最终仍在 GPU 上计算
中间过程难插桩、难复盘

vllm-xtu-moe

CPU 真正计算专家

逐层统计专家路由与激活
权重进 GPU 前可插入审计
C++ / AVX-512 内核可改造

现有方案各有侧重，空白点在右上角

开源、CPU 真算、服务器级 AMD 平台、同机性能对标。



02

怎么做

把 MoE 层劈开，再用工程手段补回路径开销。

vllm-xtu-moe · 刘锋 · 2026-09-15

系统架构：注意力留在 GPU，专家计算在 CPU

模型并行的边界清晰，数据交换集中在少数几条路径。



mapped flag + cuStreamWriteValue32 / WaitValue32 + 常驻 worker

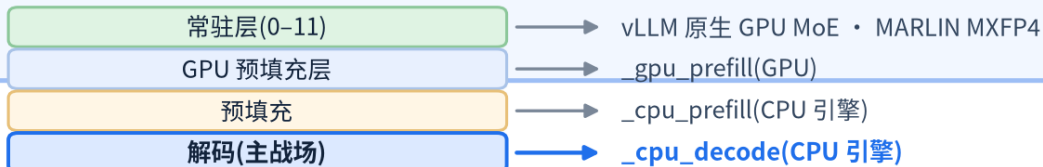
系统架构：注意力留在 GPU，专家计算在 CPU

系统架构:把 MoE 层劈开 —— 注意力/KV 留 GPU,专家计算放 CPU

vLLM fork —— 编排链(与参考实现逐字节同源,只差 1 个空白 hunk)

Scheduler · PagedAttention · CUDA Graph(FULL_DECODE_ONLY) · DSpark 投机 · Prefix Cache

RoutedExperts.forward() —— 按层分派



↓ 7 个指针 + stream

xiaotu_moe —— 自研 CPU 引擎(C++ / AVX-512,无 nvcc 依赖)

- MXFP4 权重常驻主机内存;按 NUMA 节点 mbind 切片(实测 128/128 rc=0)
- 进程内共享工作线程池,每 CCD 4 核(实测 4→8 核/CCD 无增益)
- 跨 rank EP 归约走 /dev/shm(不走 NCCL;每层 4~6.5 ms → ~14 μs)
- 异步握手:常驻 worker + mapped flag + 流内存操作,取代 cudaLaunchHostFunc
- FP4 解码:fp32 LUT + vpermps(每 32 权重 19 → 10 条指令)

TP=2 两 rank

rank 0

socket 0 · 12 CCD · 48 线程

rank 1

socket 1 · 12 CCD · 48 线程

只读本地内存
跨 socket 仅做归约

五条技术线，分别解决编排、计算、本地性和派发

每条技术线都有对应的实测结果。

T1 编排移植 参考链路完整移植进 vLLM fork 只换引擎	T2 GPU 常驻层 每层 -0.70 ms/token (C=1) 显存换延迟	T3 CPU 内核 PERMV: $19 \rightarrow 10$ 条 指令 Zen4 / AVX-512	T4 NUMA 与线程 每 rank 一个 socket 4-5 核 / CCD	T5 Marshalling V: $6.53 \rightarrow 1.80$ ms 流内异步握手
---	---	---	--	---

03

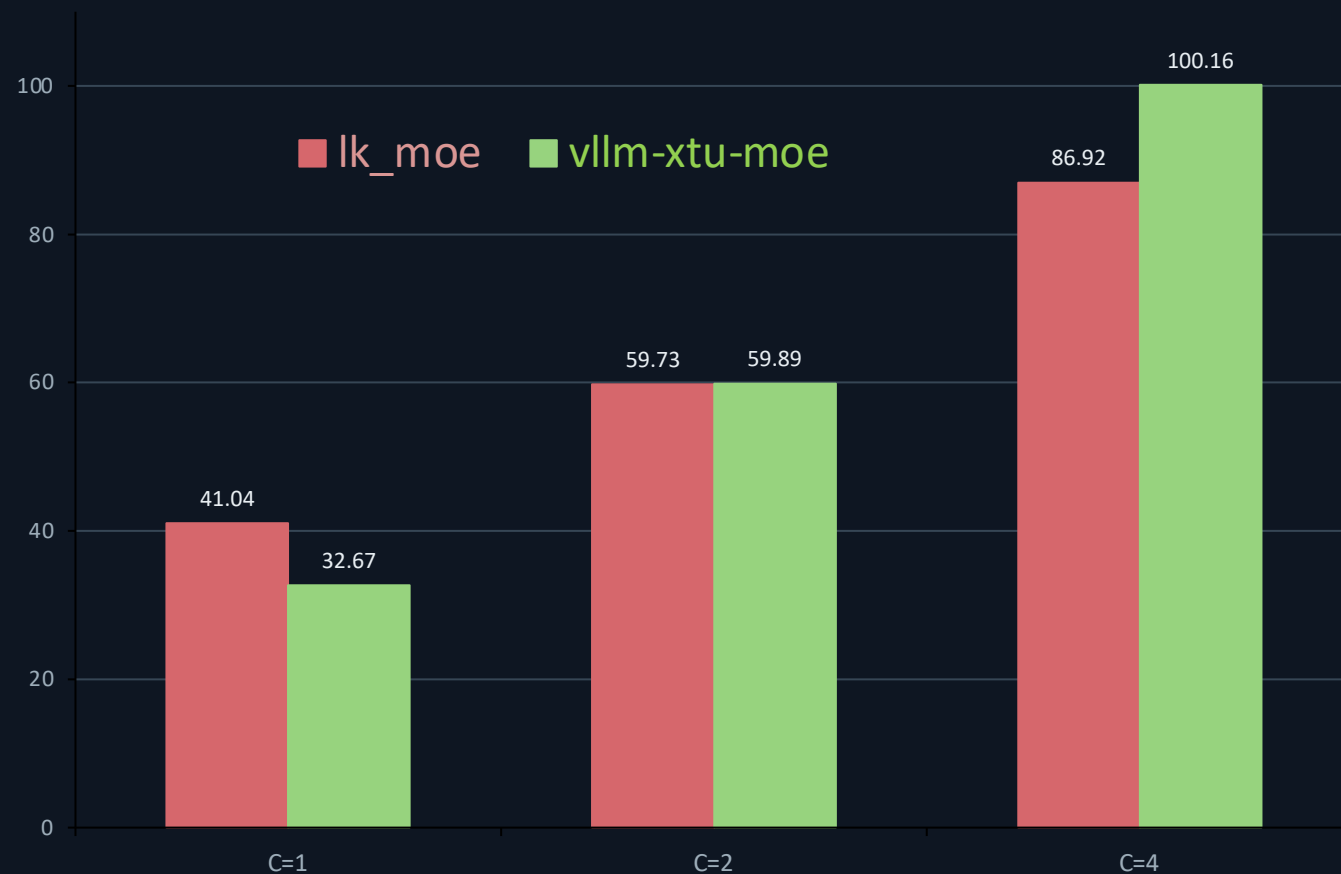
结果如何

同模型、同输出、差别是速度/成本。

vllm-xtu-moe · 刘锋 · 2026-09-15

并发越高，边际成本优势越明显

C=2 打平；C=4 聚合吞吐反超 15%。



关键读数

35.5 tok/s

单流解码

59.89 t/s

C=2 聚合

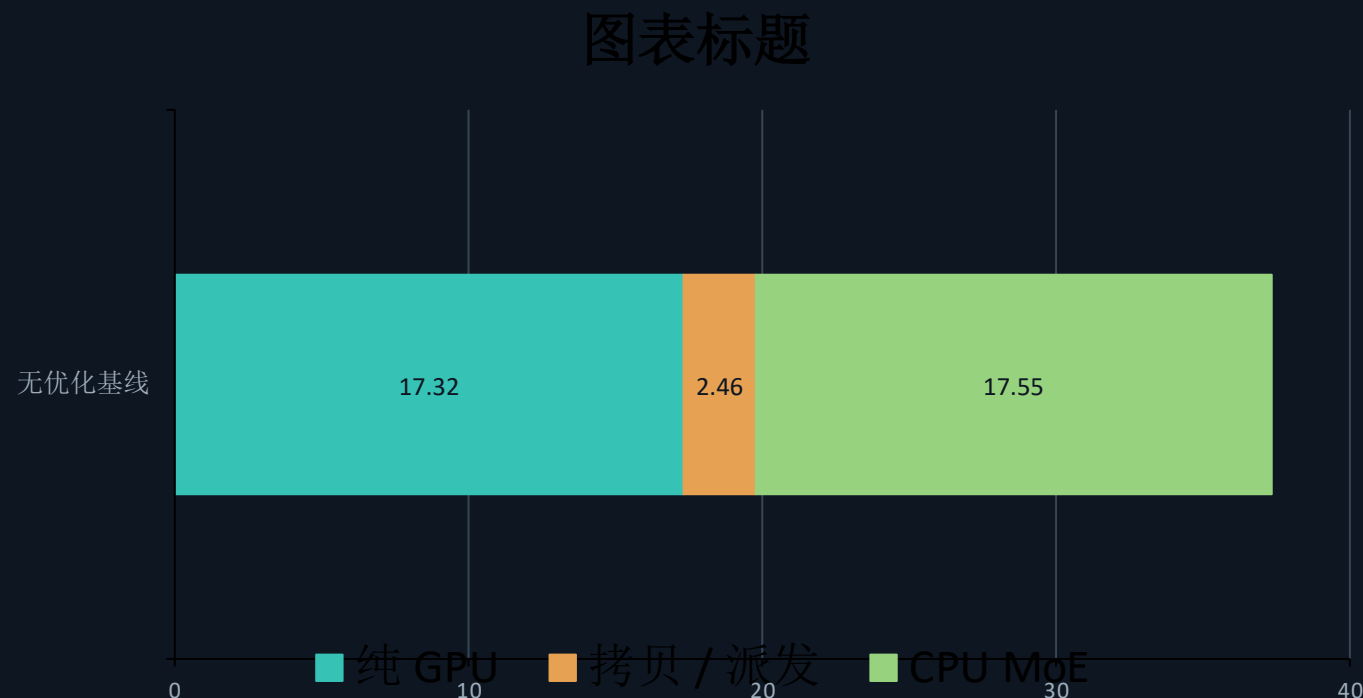
100.16 t/s

C=4 聚合

C=4: 1.15× 同机专有引擎

先做分解，再做优化：37.33 ms 的钱花在哪里

三段严格相加；CPU MoE 与派发是主要成本。



异步握手的收益

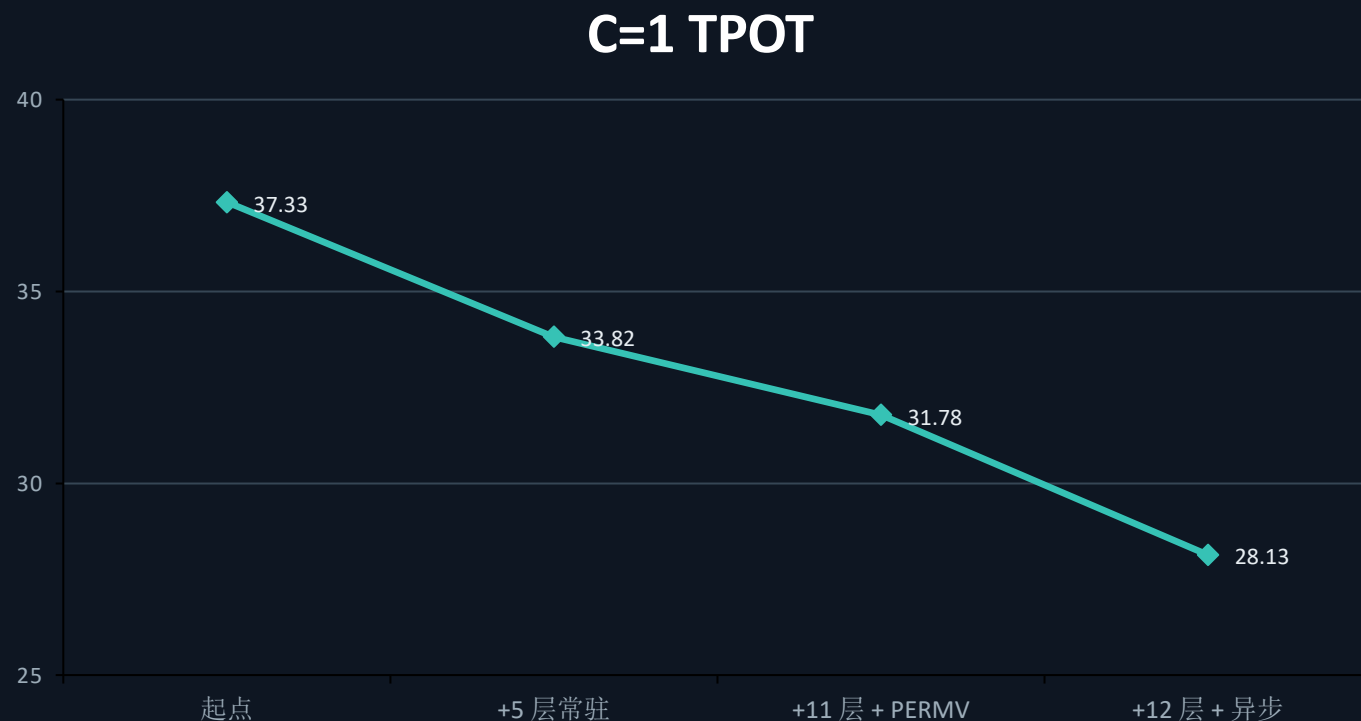
V: 6.53 → 1.80 ms

host-func 每层排空整条流；
mapped flag 让 GPU / CPU 并行推进。

C=4 聚合: 71.8 → 100.2 t/s

性能跃迁来自系统级转折，而不是单条指令

常驻层 + 异步握手，带来本轮最大的收益。



累计变化

C=1 TPOT -25%

C=2 聚合 +75%

C=4 聚合 +195%

局部指令优化只换来约 10%
compute 改善；真正跃迁来自放置
与编排。

正确性与适用边界，和性能一起验收

把可以承诺的范围写清楚，方便复现和复用。

正确性

■	u32 dump	host-func / async 逐位全等
■	Greedy 文本	5 / 5 prompt 完全一致
■	CUDA 错误	启动与运行期：0
■	数值门禁	OK=7 BAD=1，与基线相同

适用范围

- C=1–4 的单人 / 小组交互
- 私有化、离网、可审计部署
- 不追求高并发服务
- 不承诺与闭源引擎逐字节输出契约一致

04

经验与下一步

AI 负责重实验，人负责方向、判据和取舍。

vllm-xtu-moe · 刘锋 · 2026-09-15

AI 的优势是实验密度，人类的优势是换搜索空间

四个决定性输入，直接改变了优化方向。

AI 负责

分钟级微基准
批量实验与记录
局部实现与回归

人类给出

何时结案
参考数字与量级
架构放置与硬约束

破局结果

停止 30 轮局部死循环
发现投机模型应全放 GPU
把接受率目标校准到生产量级

结论：方向、判据、地面真值和取舍，不能从局部性能数据里自动推出来。

九条铁律，把试错沉淀成可复用约束

最有价值的不是某个旋钮，而是知道哪些旋钮不该再拧。

R1–R5

每 CCD 只用 4–5 核

每 rank 只读本地 socket 内存

先量分解，再谈优化

先算 roofline，再谈目标

只信 numa_maps 实测

R6–R9

同一协议、同一负载、同一口径

草稿模型永远全放 GPU

mapped flag 必须构造期分配

负结果写入台账

优先复用，再自研

路线图：从“能跑”走向“可复用”

后续重点是固定成本、显存再平衡，以及更容易跟随 vLLM 主线的接口形态。

近期 1-2 月

P0 / P1

CPU 每层 0.19 ms; 常驻层 14+; C=1
追平专有引擎

中期 3-6 月

全场景

恢复 GPU 预填充; 投机自适应; host
内存去重

在同级别更新硬件平台上实现双倍性能

长期 6-12 月

解耦式推理

CPU 专家池 + GPU 注意力池; 跨机扩展

长期愿景：把“可观测、可改造、可审计”的异构推理能力，沉淀为可复用的 vLLM 外部 MoE backend。

我们证明了一件事

买不起 GPU 集群的团队，也能在自己的服务器上运行满血最新超大模型。

性能满足科研、个人/小组生产需求

在 Harness / Agent Loop支持下，长程缓冲命中率>99%，本项目就是自我支持的开发过程可观测，代码可改造，部署可自主控制

使用低成本DRAM容纳更大的deepseek-v4.1-flash模型（500B参数）乃至万亿参数模型

谢谢

刘锋 · 2026-09-15